In [21]:
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import os
os.environ["OMP_NUM_THREADS"] = "1"
```

In [22]:
```python
df = pd.read_csv("Mall_Customers.csv")
```

In [23]:
```python
df.head()
```

Out[23]:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

In [24]:
```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

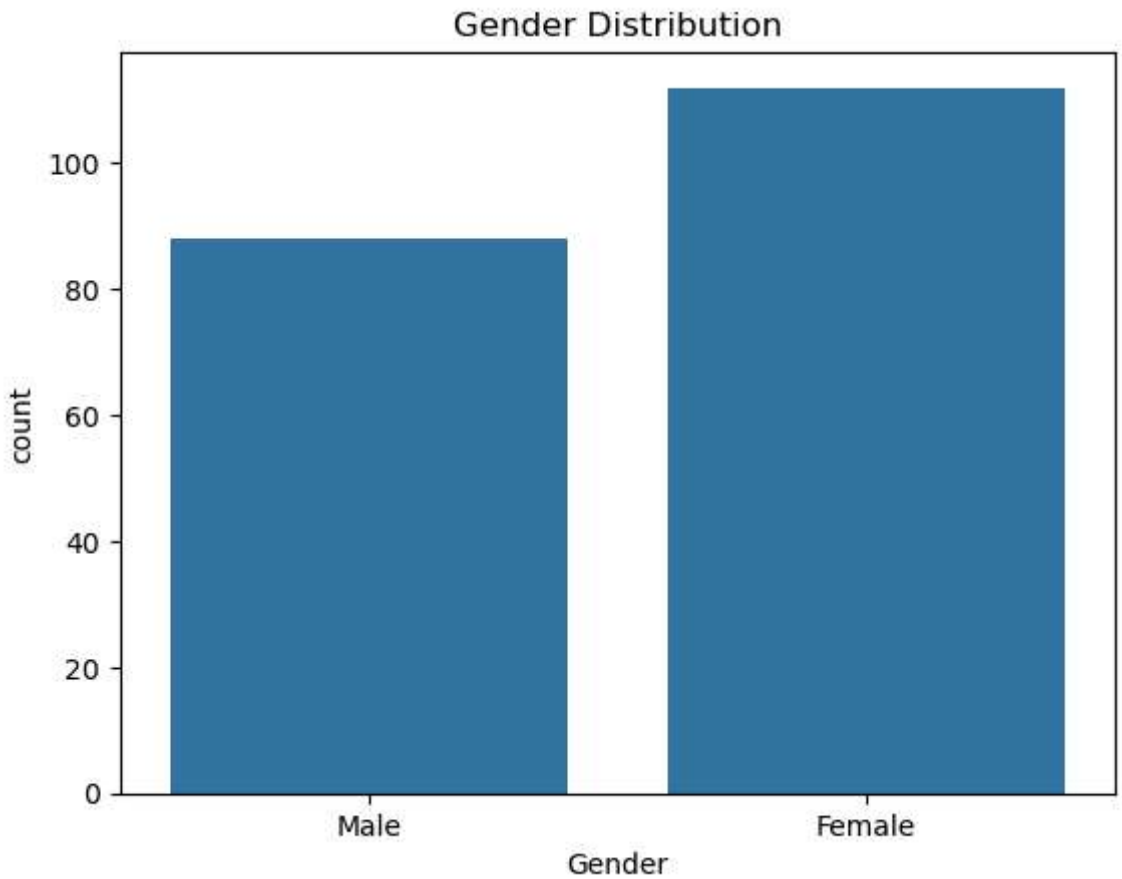
In [25]:
```python
print(df.describe())
```

```
       CustomerID         Age  Annual Income (k$)  Spending Score (1-100)
count  200.000000  200.000000          200.000000              200.000000
mean   100.500000   38.850000           60.560000               50.200000
std     57.879185   13.969007           26.264721               25.823522
min      1.000000   18.000000           15.000000                1.000000
25%     50.750000   28.750000           41.500000               34.750000
50%    100.500000   36.000000           61.500000               50.000000
75%    150.250000   49.000000           78.000000               73.000000
max    200.000000   70.000000          137.000000               99.000000
```

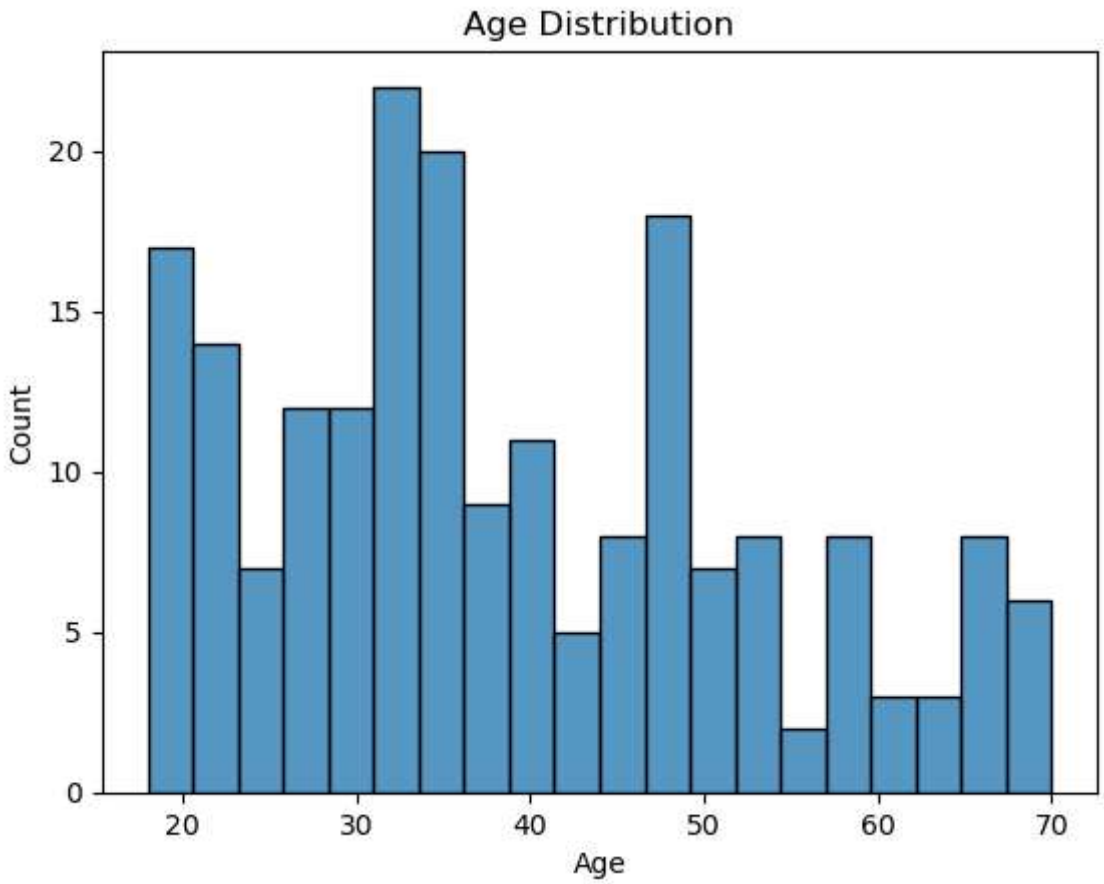
In [26]:
```python
print(df.isnull().sum())
```

```
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64
```

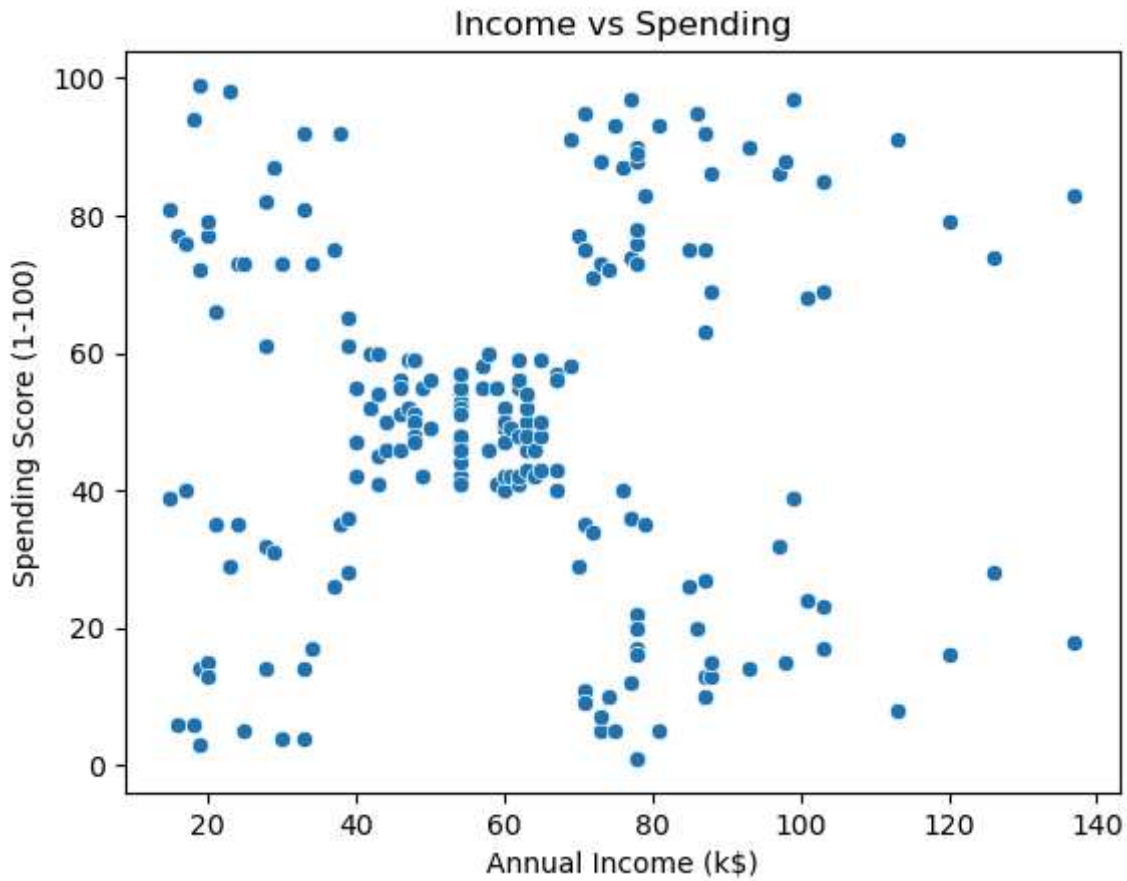
In [27]:
```
sns.countplot(x='Gender', data=df)
plt.title("Gender Distribution")
plt.show()
```



In [28]:
```
sns.histplot(df['Age'], bins=20)
plt.title("Age Distribution")
plt.show()
```



In [29]:
```
sns.scatterplot(x='Annual Income (k$)', y='Spending Score (1-100)', data=df)
plt.title("Income vs Spending")
plt.show()
```

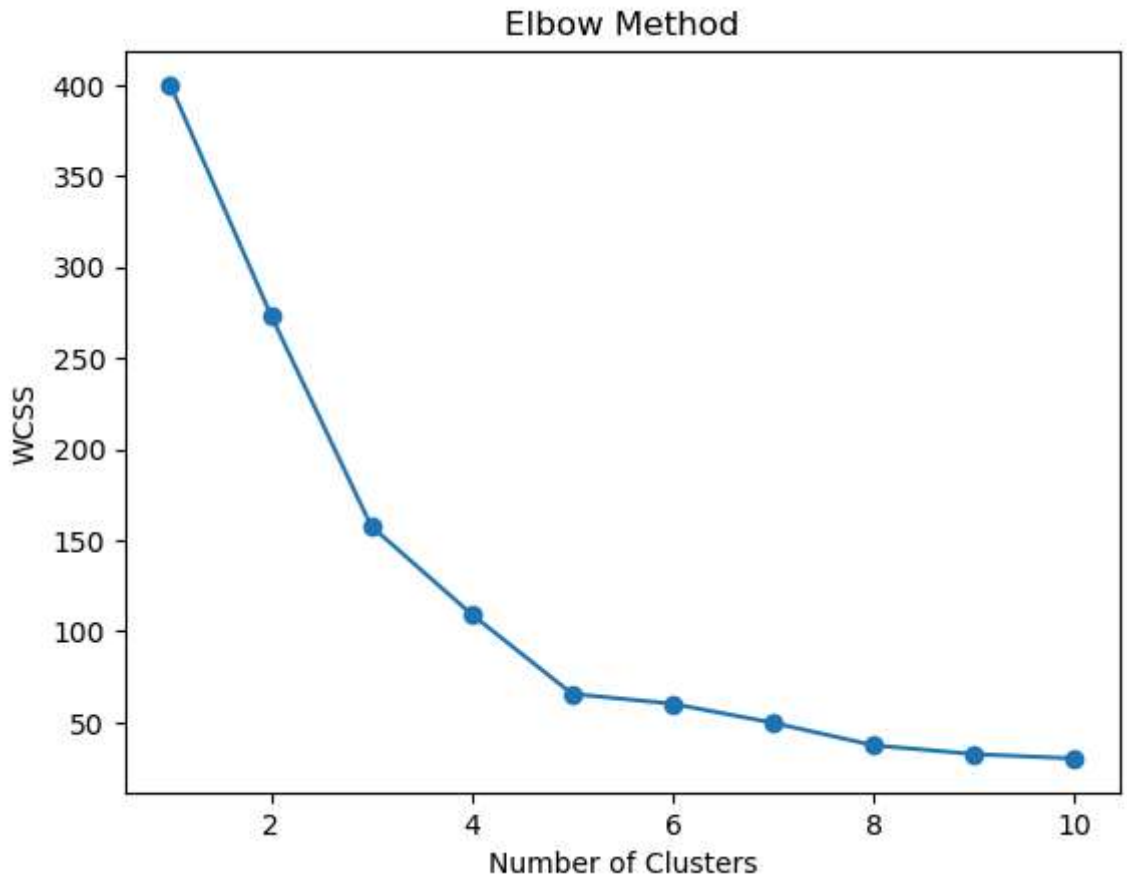


In [30]:
```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
```

In [31]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [32]:
```
wcss = []

for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.plot(range(1, 11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()
```

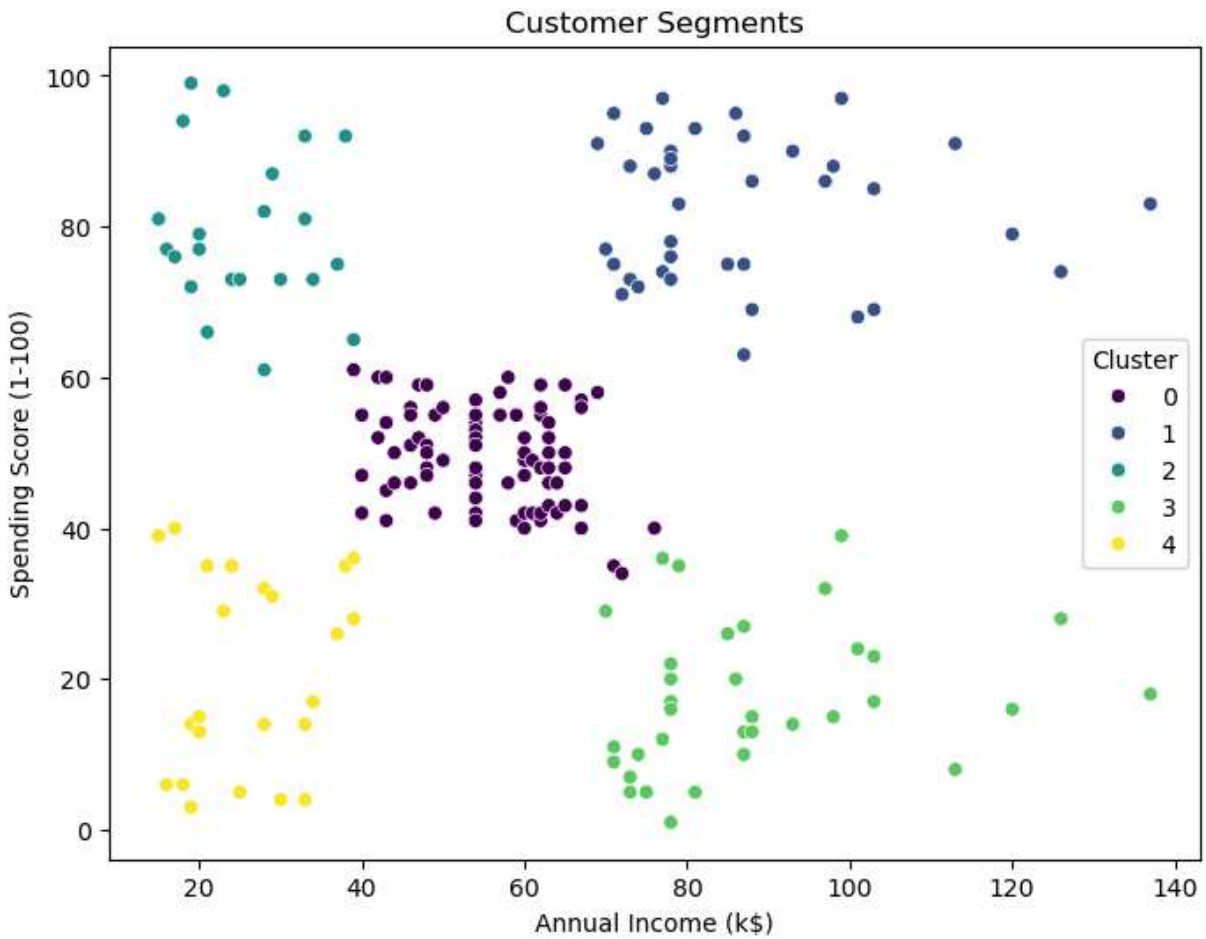
```
C:\Users\GADDAM AKASH\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: U
serWarning: KMeans is known to have a memory leak on Windows with MKL, when there ar
e less chunks than available threads. You can avoid it by setting the environment va
riable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\GADDAM AKASH\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: U
serWarning: KMeans is known to have a memory leak on Windows with MKL, when there ar
e less chunks than available threads. You can avoid it by setting the environment va
riable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\GADDAM AKASH\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: U
serWarning: KMeans is known to have a memory leak on Windows with MKL, when there ar
e less chunks than available threads. You can avoid it by setting the environment va
riable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\GADDAM AKASH\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: U
serWarning: KMeans is known to have a memory leak on Windows with MKL, when there ar
e less chunks than available threads. You can avoid it by setting the environment va
riable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\GADDAM AKASH\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: U
serWarning: KMeans is known to have a memory leak on Windows with MKL, when there ar
e less chunks than available threads. You can avoid it by setting the environment va
riable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\GADDAM AKASH\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: U
serWarning: KMeans is known to have a memory leak on Windows with MKL, when there ar
e less chunks than available threads. You can avoid it by setting the environment va
riable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\GADDAM AKASH\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: U
serWarning: KMeans is known to have a memory leak on Windows with MKL, when there ar
e less chunks than available threads. You can avoid it by setting the environment va
riable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\GADDAM AKASH\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: U
serWarning: KMeans is known to have a memory leak on Windows with MKL, when there ar
e less chunks than available threads. You can avoid it by setting the environment va
riable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\GADDAM AKASH\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: U
serWarning: KMeans is known to have a memory leak on Windows with MKL, when there ar
e less chunks than available threads. You can avoid it by setting the environment va
riable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\GADDAM AKASH\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: U
serWarning: KMeans is known to have a memory leak on Windows with MKL, when there ar
e less chunks than available threads. You can avoid it by setting the environment va
riable OMP_NUM_THREADS=1.
  warnings.warn(
```



In [34]:
```
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X_scaled)
```

```
C:\Users\GADDAM AKASH\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: U
serWarning: KMeans is known to have a memory leak on Windows with MKL, when there ar
e less chunks than available threads. You can avoid it by setting the environment va
riable OMP_NUM_THREADS=1.
  warnings.warn(
```

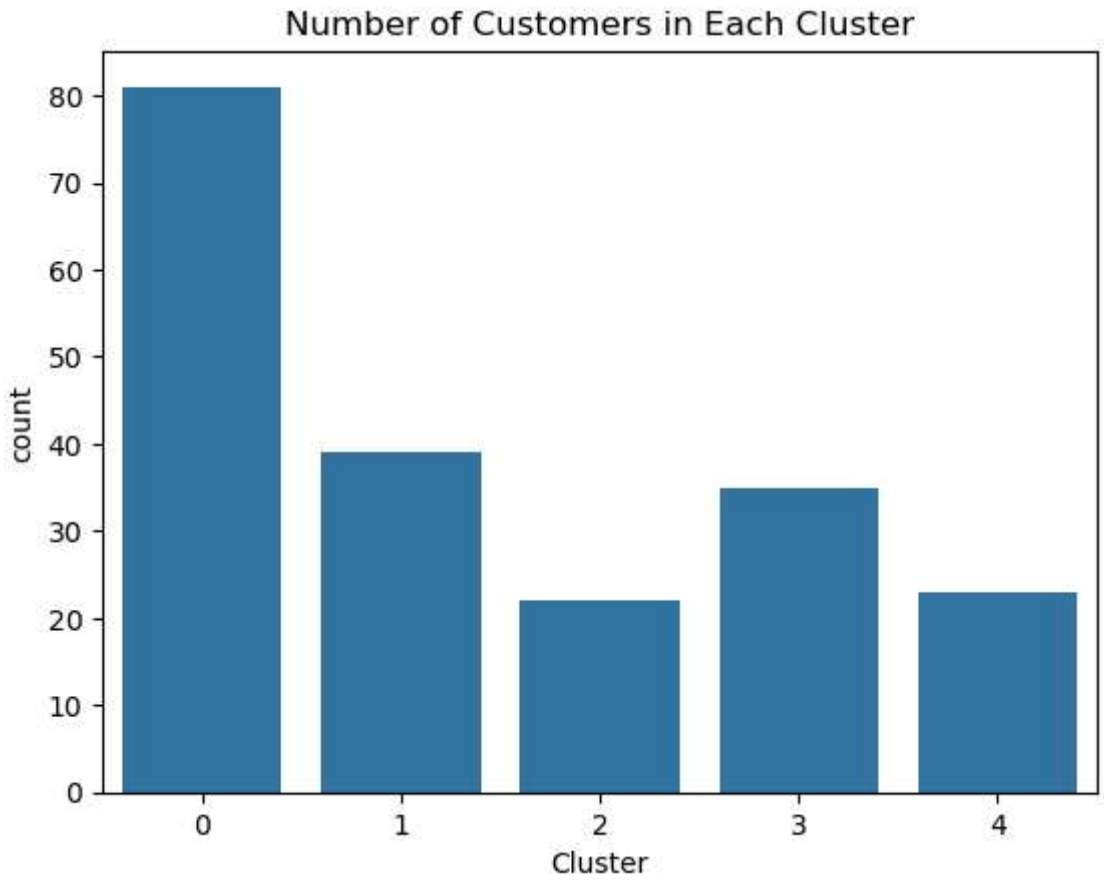
In [35]:
```
plt.figure(figsize=(8,6))

sns.scatterplot(
    x=df['Annual Income (k$)'],
    y=df['Spending Score (1-100)'],
    hue=df['Cluster'],
    palette='viridis'
)

plt.title("Customer Segments")
plt.show()
```



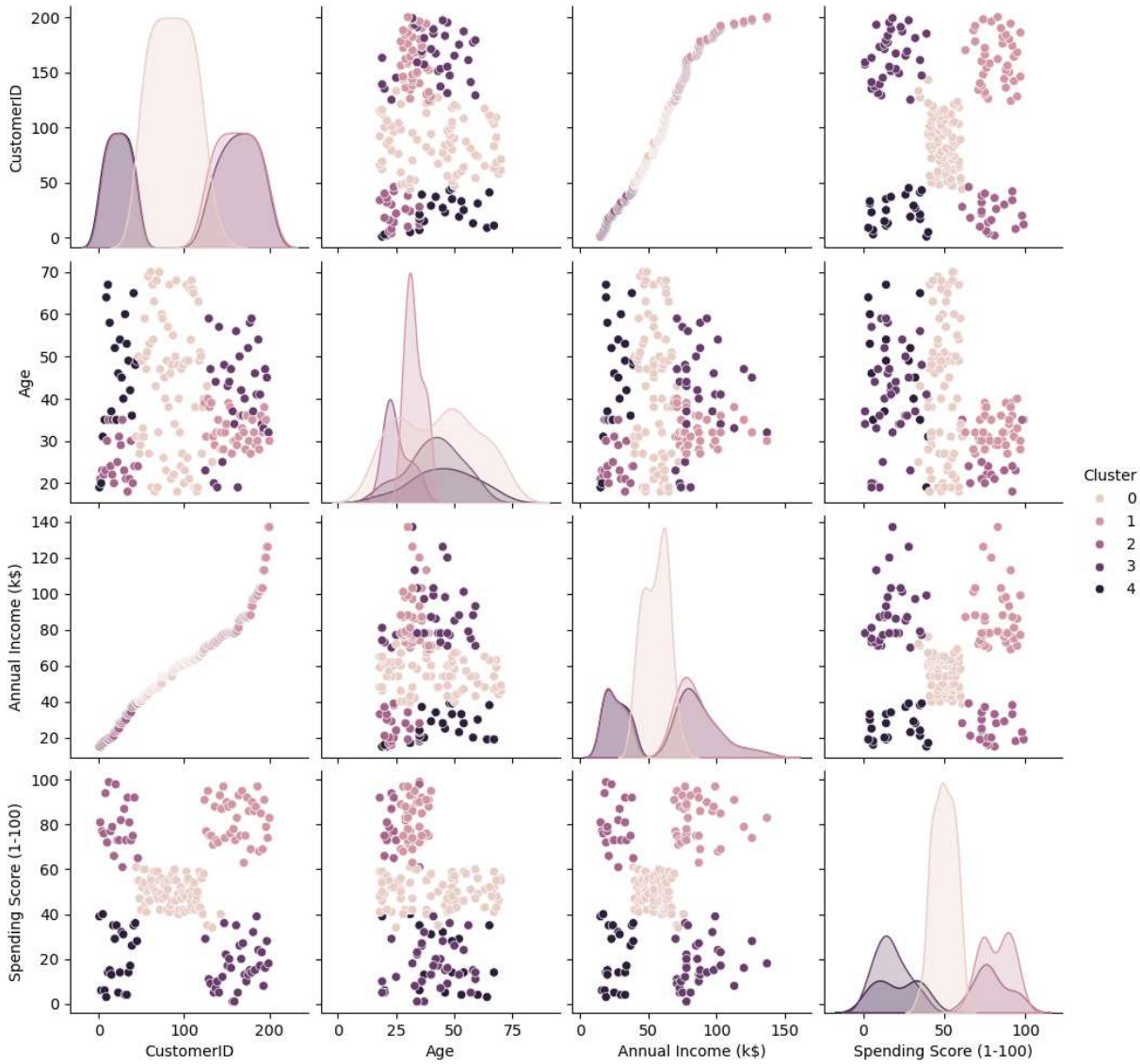
In [36]:
```
cluster_summary = df.groupby('Cluster').mean(numeric_only=True)
print(cluster_summary)
```

```
         CustomerID        Age  Annual Income (k$)  Spending Score (1-100)
Cluster
0         86.320988  42.716049           55.296296               49.518519
1        162.000000  32.692308           86.538462               82.128205
2         23.090909  25.272727           25.727273               79.363636
3        164.371429  41.114286           88.200000               17.114286
4         23.000000  45.217391           26.304348               20.913043
```

In [37]:
```
sns.countplot(x='Cluster', data=df)
plt.title("Number of Customers in Each Cluster")
plt.show()
```



In [39]:
```
sns.pairplot(df, hue='Cluster')
plt.show()
```


CustomerID
Age
Annual Income (k$)
Spending Score (1-100)
Cluster
0
1
2
3
4

In [ ]: